then the constructor is called, in this case assigning an empty object to the `pages` property of the new object.

The example in listing 4.8 is similar to that in listing 4.7. The conceptual difference is that instead of creating an object out of another object, you create an object out of a class. This means you've now learned three ways to create objects in CoffeeScript: ex nihilo, using another object, and using a class.

If you're wondering at this stage how classes and constructors relate to the underlying JavaScript, then try compiling the examples in this section and having a look. The veil will be lifted in chapter 5, where the relationship of CoffeeScript syntax to the JavaScript object model is explored in detail.

4.8.2. Object identity

Objects are never equal to each other; an object is equal only to itself:

```
businessViews is personalViews
# false
```

Equality will be true for variables referencing objects only if they refer to the same object:

```
objectReferenceOne = {}
objectReferenceTwo = objectReferenceOne
objectReferenceTwo is objectReferenceOne
# true
```

Two objects created from the same class are not equal:

```
class Black
green = new Black
```

Now `green` is a `new Black` but it is never *the* `new Black`:

```
green is new Black
# false
```